

Front-End React T1

Projeto Avaliativo - Módulo 1 - Semana 13

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	2
4. REQUISITOS DA APLICAÇÃO	3
4.1. FORMATO DO SISTEMA	3
4.2. REQUISITOS FUNCIONAIS (RF)	4
4.3. REQUISITOS TÉCNICOS (O QUE DEMONSTRAR DO SEMESTRE)	6
4.4. USO DE IA (COM VALIDAÇÃO OBRIGATÓRIA)	7
5. ROTEIRO DA APLICAÇÃO	8
5.1. FORMATO DO SISTEMA	9
5.2. DOCUMENTAÇÃO NO README.MD	9
5.3. USO DO GITHUB	10
5.4. GRAVAÇÃO DE VÍDEO	10
6. CRITÉRIOS DE AVALIAÇÃO	10
7. CHECKLIST FINAL DE ENTREGA	12

1. CONTEXTUALIZAÇÃO

O mercado de front-end não contrata quem "sabe um pouco de cada coisa", contrata quem junta tudo numa aplicação que funciona: uma interface bonita e responsiva (funciona no celular e no desktop), acessível (qualquer pessoa consegue usar, inclusive com leitor de tela) e encontrável pelo Google (SEO); que reage ao que o usuário faz (DOM e eventos), lembra das informações dele (persistência) e busca dados de fora (rede). Tudo isso versionado no Git, organizado num Kanban e documentado num README, porque software é trabalho em equipe e comunicação técnica.

No Mini-Projeto (Semana 06) você construiu o SkillMatch JS: um motor, em JavaScript puro rodando no console, que comparava as habilidades de um candidato com os requisitos de vagas de front-end júnior e calculava a compatibilidade. Era lógica pura, sem tela.

Agora chegou a hora de dar uma cara a esse motor. Você vai transformar o SkillMatch num produto web completo, o mesmo cérebro de antes, mas dentro de uma aplicação real que qualquer pessoa abre no navegador, preenche o próprio perfil, vê as vagas com seu percentual de compatibilidade e recebe uma recomendação de estudo. É o projeto que fecha o Módulo 01 e o primeiro item de peso do seu portfólio.

2. DESAFIO

Você foi efetivado na startup de recrutamento onde fez o protótipo do SkillMatch. O time gostou do motor de análise, mas pediu: "o RH não vai abrir o console do navegador para usar isso, precisamos de uma tela de verdade, que rode em qualquer dispositivo, lembre do perfil de quem usou e puxe as vagas de uma fonte de dados."

Sua missão é evoluir o SkillMatch de script de console para uma aplicação web (Single Page) em HTML, CSS e JavaScript puro, reaproveitando o motor de compatibilidade que você já escreveu e cobrindo tudo o que foi trabalhado no Módulo 01.

Ao construir a aplicação, você coloca em prática os aprendizados em:

- Lógica e JavaScript (S02–S06): tipos, condicionais, operadores, laços, funções, arrow functions, arrays e seus métodos, objetos, POO (classes, herança, this), callbacks, closures, Promises e async/await, o motor do SkillMatch.
- HTML e CSS (S07/S10): marcação, CSS externo, Flexbox, box model, e responsividade mobile-first (a página funciona em qualquer tela).
- DOM e eventos (S08/S09): capturar formulário, validar entradas, reagir a eventos e desenhar a tela com JavaScript (createElement/classList).
- Semântica, acessibilidade e SEO (S11): tags que significam algo, label/for, alt, foco visível, aria-label, title/meta description.
- Persistência e rede (S11): localStorage para lembrar do usuário e fetch + async/await para carregar as vagas, tratando os três estados (carregando/vazio/erro).
- Organização e ferramentas (S12): dividir o JavaScript em módulos ES (import/export), usar o ecossistema npm como ferramenta de apoio (opcional).
- Processo profissional: Git/GitHub (branches, commits descritivos), Trello (Kanban) e README, tratar suas alterações como algo que impacta o projeto inteiro.

3. RESULTADOS ESPERADOS (ENTREGA)

Este é um projeto individual. Ao final, você deverá entregar:

- Um repositório no GitHub público (para acesso via link) com a aplicação rodando (HTML + CSS + JS).
- Um arquivo README.md documentando o projeto (ver 5.2).
- Um histórico de commits e branches que conte a evolução do trabalho (ver 5.3).
- Um vídeo de até 7 minutos demonstrando o funcionamento e explicando as decisões (ver 5.4).
- Todos os links devem ser submetidos no AVA.

Links obrigatórios. O(a) estudante deverá enviar no AVA:

- link do repositório público do projeto no GitHub;
- link de acesso ao quadro Kanban utilizado no projeto (Trello ou similares). Acessível via link;
- link de acesso ao vídeo apresentação do projeto (ser possível de visualizar via link). Via Google Drive ou YouTube como vídeo "não listado";

● Prazo de entrega: **20/07/26, segunda, até às 22h**

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. Não esqueça de submeter todos links no AVA. Os links não submetidos terão penalidade na nota.
3. Projetos com plágio (cópia da internet ou de colegas) recebem nota 0. Você pode consultar materiais, documentação e exemplos e até pedir ajuda à IA (ver 4.4), desde que adapte ao desafio e saiba explicar cada linha com segurança.

4. REQUISITOS DA APLICAÇÃO

4.1. FORMATO DO SISTEMA

A aplicação é estática (HTML/CSS/JS puro) e roda no navegador — não precisa de servidor nem de banco de dados. Sugestão de estrutura (você pode adaptar, desde que fique organizada):

```

skillmatch-web/
├── index.html           # a página (HTML semântico)
├── README.md
├── assets/              # MESMA organização das aulas (S09–S11)
│   ├── styles/
│   │   └── index.style.css # estilos + responsividade (mobile-first)
│   ├── scripts/
│   │   ├── main.js        # ponto de entrada (<script type="module">)
│   │   └── motor.js        # o MOTOR do SkillMatch (engine do mini):
│   │                       compatibilidade classes — export
│   ├── ui.js             # tela: render dos cards, formulário, DOM/eventos — export
│   └── dados.js           # carregar vagas (fetch) + localStorage — export
├── dados/
│   └── vagas.json         # catálogo de vagas (carregado via fetch)
├── img/
│   └── logo.svg           # logo e imagens (ícones em img/ ou ícones/)

```

É a mesma estrutura das aulas (S09–S11): index.html na raiz e tudo dentro de assets/ (styles/, scripts/, mais dados/ e img/). No HTML, referencie como você já faz.

A novidade vem da S12: o main.js agora é type="module" e puxa os outros módulos, que são vizinhos dele: import { ... } from "../motor.js". Atenção a um detalhe: o caminho do import é relativo ao arquivo .js (./motor.js), mas o do fetch é relativo à página, de dentro do dados.js, busque fetch("../assets/dados/vagas.json").

💡 Os 3 módulos (motor = regras · ui = tela · dados = dados/persistência) são a aplicação direta da divisão dados × regras × tela, orquestrados pelo main.js (o fluxo), exatamente a ideia de módulos da S12. Eles se ligam por import/export.

⚠️ Como rodar (importante): porque o projeto usa módulos ES e fetch, ele não funciona abrindo o arquivo direto (file://), abra com o Live Server do VS Code (ou outro servidor local). localhost é "contexto seguro", igual ao que você viu na S11.

4.2. REQUISITOS FUNCIONAIS (RF)

Os RF estão agrupados por camada. Os grupos A–D são obrigatórios (cobrem o semestre inteiro). Os itens marcados ★ são bônus (valem como diferencial — ver 4.3 e seção 6).

Grupo A — O motor SkillMatch (reaproveitado do mini · S02–S06)

- RF01 — Perfil do candidato: representar o candidato como objeto (nome, area, habilidades em array, experienciaMeses). Dê um uso ao experienciaMeses (ex.: exibir no card de perfil e/ou usá-lo como critério de desempate entre vagas de mesma compatibilidade). Diferente do mini, o perfil agora é preenchido pelo usuário no formulário (RF10) e persistido (RF14).
- RF02 — Catálogo de vagas: uma lista (array) de pelo menos 4 vagas, cada uma um objeto (id, empresa, cargo, requisitos, salario, modalidade). As vagas são carregadas via fetch (RF13), não escritas à mão no HTML.
- RF03 — Cálculo de compatibilidade: para cada vaga, calcular o percentual (requisitos atendidos ÷ total de requisitos × 100) e separar habilidades encontradas × faltantes.
- RF04 — Classificação: classificar cada vaga (Alta 80–100% · Média 50–79% · Baixa 0–49%) usando if/else, switch ou ternário.
- RF05 — Melhor vaga + recomendação de estudo: identificar a vaga de maior compatibilidade e gerar uma recomendação com base nas habilidades que mais faltam.
- RF06 — Métodos de array: usar pelo menos 3 entre map, filter, find, every, reduce (ex.: map para gerar os resultados, filter para faltantes, reduce para a melhor vaga).
- RF07 — POO: ter pelo menos uma classe (ex.: Vaga) com construtor, atributos e método usando this (ex.: o próprio cálculo de compatibilidade como método, usando this.requisitos), e uma classe que herda de outra com um motivo claro — ex.: VagaFrontEnd extends Vaga que acrescenta um atributo (stack ou senioridade) e sobrescreve um método (um rótulo de exibição, ou um peso extra em certos requisitos). A herança precisa de um propósito que você saiba explicar (não "só para cumprir a tabela").
- RF08 — Callback e closure: usar pelo menos um callback (função que recebe outra função) e uma closure (ex.: um contador de quantas análises foram feitas na sessão).

💡 Como o JSON vira objeto do motor (liga RF13 → RF06 → RF07): depois do fetch, use map para transformar cada objeto do vagas.json numa instância de Vaga (new Vaga(...)). Assim os dados (JSON) viram regras (o motor com seus métodos), e o cálculo de compatibilidade fica dentro da classe (this.requisitos) — não como função solta nem com as vagas escritas à mão.

Grupo B — Interface (S07 · S08 · S09 · S10 · S11)

- RF09 — HTML semântico, acessível e com SEO: estruturar a página com landmarks (header/nav/main/section/footer), um único h1 e hierarquia de títulos, label/for nos campos, aria-label onde fizer sentido, foco visível e lang no html. Inclua ao menos uma imagem (ex.: um logo no header com alt informativo e/ou um ícone decorativo com alt="") para exercitar a distinção decorativo × informativo. Incluir SEO on-page: title descritivo e meta description. (Rode o Lighthouse para conferir Acessibilidade/SEO, como na S11.)
- RF10 — Formulário de perfil com validação e eventos: um formulário onde o usuário informa nome, área e habilidades (e, opcionalmente, o tempo de experiência — ver RF01). Capturar com addEventListener, impedir o reload com preventDefault, e validar as entradas (campos obrigatórios; feedback de erro acessível). É o "JS marca, CSS pinta" da S09.
- RF11 — Render dinâmico no DOM: os cards de vaga devem ser gerados por JavaScript a partir dos dados (createElement/classList), não escritos à mão no HTML. Cada card mostra empresa, cargo, % de compatibilidade, classificação, habilidades encontradas e habilidades faltantes (como no resultado do mini). Os cards se reorganizam em colunas com Flexbox (flex-wrap + gap).
- RF12 — Layout responsivo (mobile-first): meta viewport, unidades relativas/fluidas (% , rem, vw/clamp()), media queries e mídia fluida (max-width: 100%, object-fit). Layout em Flexbox. A página tem que funcionar bem do celular ao desktop (testar no modo responsivo do DevTools).

Grupo C — Dados e persistência (S11)

- RF13 — fetch das vagas com os 3 estados: carregar o catálogo de vagas com fetch + async/await a partir de um assets/dados/vagas.json local (caminho recomendado). Tratar explicitamente os três estados da S11: carregando (feedback "Carregando vagas..."), vazio (nenhuma vaga / nenhum resultado → "Nada encontrado") e erro (a rede falhou → mensagem clara). O caminho normal — sucesso (renderizar os cards) — não é um dos três tratamentos. Usar try/catch, checar response.ok e anunciar a mensagem de erro/vazio com aria-live. Lembre o conceito cliente-servidor do mini: o fetch é o cliente pedindo dados — a Promise que você antes simulava agora é rede de verdade. ★ Opcional (avançado): usar uma API pública GET no lugar do JSON — só se ela devolver as vagas no formato esperado (com o array requisitos) e sem barreira de CORS; senão, fique no vagas.json local.
- RF14 — Persistência com localStorage: lembrar do perfil do candidato (e/ou preferências, como uma ordenação) entre recarregamentos — setItem/getItem, JSON.stringify/JSON.parse, e tratar o null da primeira visita. Reforce a fronteira de segurança: nunca guardar dados sensíveis.

Grupo D — Organização e qualidade (S12)

- RF15 — Módulos ES: organizar o JavaScript em módulos com import/export e <script type="module">, separando motor (regras) × ui (tela) × dados (fetch/localStorage). Nada de um único .js gigante.
- RF16 — Código limpo: usar nomes de variáveis, funções, classes e ids que façam sentido com o que está relacionado no bloco de código.

★ Bônus (diferencial — não obrigatórios)

- ★ Browser API: "clima/contexto onde estou" com Geolocation (tratando permissão negada), ou outra API nativa.
- ★ Deploy no GitHub Pages e link no README (serve via https — também resolve o file://).
- ★ Tema claro/escuro persistido no localStorage.
- ★ Filtro/ordenação de vagas (por modalidade, salário ou compatibilidade).
- ★ npm como ferramenta: um package.json com um script de servidor local (ex.: live-server) — usando o que você viu na S12. (A aplicação continua rodando sem Node; isto é só conforto de dev.)

4.3. REQUISITOS TÉCNICOS (O QUE DEMONSTRAR DO SEMESTRE)

Conteúdo (semana)	Obrigatório?	Como demonstrar
Tipos, operadores, condicionais (if/else, switch, ternário) (S02–S03)	Sim	Cálculo de compatibilidade e classificação
Laços (for/while) ou métodos de array que iteram (S03–S04)	Sim	Iterar requisitos/habilidades (os métodos de array já contam; ≥1 laço explícito é bem-vindo)
Funções e arrow functions (S03)	Sim	Regras do motor; callbacks dos eventos
Arrays + métodos map/filter/find/every/reduce (S04)	Sim (≥3)	Processar vagas e habilidades
Objetos (chaves/valores) (S04)	Sim	Candidato e vagas
POO: classe, construtor, atributos, método, this, herança (S05)	Sim	Classe Vaga + VagaFrontEnd
Callback e closure (S06)	Sim	Mensagem final; contador de análises
Promises / async/await (S06)	Sim	No fetch (RF13) — agora real, não simulado
HTML semântico (S07/S11)	Sim	Landmarks, um h1, hierarquia
CSS externo, box model, Flexbox (S07/S10)	Sim	Layout dos cards e do formulário
Responsividade mobile-first (media queries, unidades fluidas) (S10)	Sim	Funciona do celular ao desktop
Especificidade/cascata (S10)	Sim	Estilos sem conflito; explicar no README se usar !important
Bootstrap / Font Awesome via CDN	★ Opcional	Se usar, só via CDN e sem npm

(S10)		install
DOM + eventos: addEventListener, createElement, classList, preventDefault (S08/S09)	Sim	Formulário + render dos cards
Validação de formulário (S09)	Sim	Campos obrigatórios + erro acessível
Acessibilidade: label/for, alt, aria-label, foco, lang (S11)	Sim	Página navegável por teclado
SEO on-page: title, meta description (S11)	Sim	No <head>
fetch + 3 estados (carregando/vazio/erro), try/catch, response.ok (S11)	Sim	Carregar as vagas
localStorage + JSON.stringify/parse + tratar null (S11)	Sim	Lembrar do perfil
Geolocation / outra Browser API (S11)	★ Opcional	"Onde estou" / contexto
Módulos ES import/export (S12)	Sim	motor/ui/dados separados
npm / package.json / script de dev (S12)	★ Opcional	Servidor local de desenvolvimento
Git: branches, commits descritivos, GitHub (S06+)	Sim	Histórico do repositório
Kanban (Trello)	Sim	Quadro do projeto
var/let/const e escopo de bloco	Sim	Priorizar const/let; explicar as escolhas (e onde o escopo importou) no README

❌ Fora do escopo do Módulo 01 (não usar): React ou qualquer framework JS (Vue/Angular), TypeScript, bundlers/build (Webpack/Vite/Babel), CSS Grid como sistema de layout (foi só menção, use Flexbox), Sass/CSS-in-JS, back-end/servidor/banco de dados, fetch com POST/PUT/DELETE que grave em servidor, jQuery/axios e bibliotecas não vistas. Tudo isso chega no Módulo 02 — aqui o mérito é fazer com o que aprendemos.

4.4. USO DE IA (COM VALIDAÇÃO OBRIGATÓRIA)

Você pode usar IA como colega de apoio em pontos específicos — mas o que conta é o que você sabe revisar e explicar. Lema do semestre: "a IA escreve o caminho feliz; o profissional cobre o caminho triste." Usar IA é opcional: todo o projeto tem caminho 100% manual, e ninguém é avaliado por ter ou não a ferramenta.

Onde a IA ajuda bem (sugestões):

- Gerar o assets/dados/vagas.json (um catálogo de vagas fictícias no formato que você definiu).
- Rascunhar uma função e você reescrever no seu estilo.
- Sugerir melhorias de acessibilidade ou de responsividade para você conferir na MDN/Lighthouse.

Como pedir (método CARTA): Contexto e papel · Ação (uma tarefa) · Requisitos (o obrigatório e o proibido) · formaTo · Aceite (como saber que ficou bom). Sempre inclua nos Requisitos: "JavaScript/HTML/CSS puro, sem framework, sem TypeScript, sem build; compatível com o que se aprende no Módulo 01."

Validação OBRIGATÓRIA — antes de aceitar qualquer trecho da IA:

1. Roda? Testei e faz exatamente o que eu pedi?
2. Está dentro do que aprendemos? Veja as  bandeiras vermelhas, se a IA trouxer qualquer uma, reescreva no jeito ensinado:

 A IA trouxe...	 Faça assim (Módulo 01)
React/JSX, Vue, Angular	HTML + DOM puro (createElement)
TypeScript (: string, interface)	JavaScript puro
import de pacote npm / Vite / Webpack	módulos ES locais (./motor.js) ou CDN
display: grid para o layout	Flexbox
axios / jQuery / \$(...)	fetch / document.querySelector
back-end, Express, banco de dados	vagas.json local + localStorage
localStorage guardando token/senha	nunca dados sensíveis

3. Eu sei explicar? Consigo justificar cada linha em voz alta? "Você não usa um código que não consegue explicar."

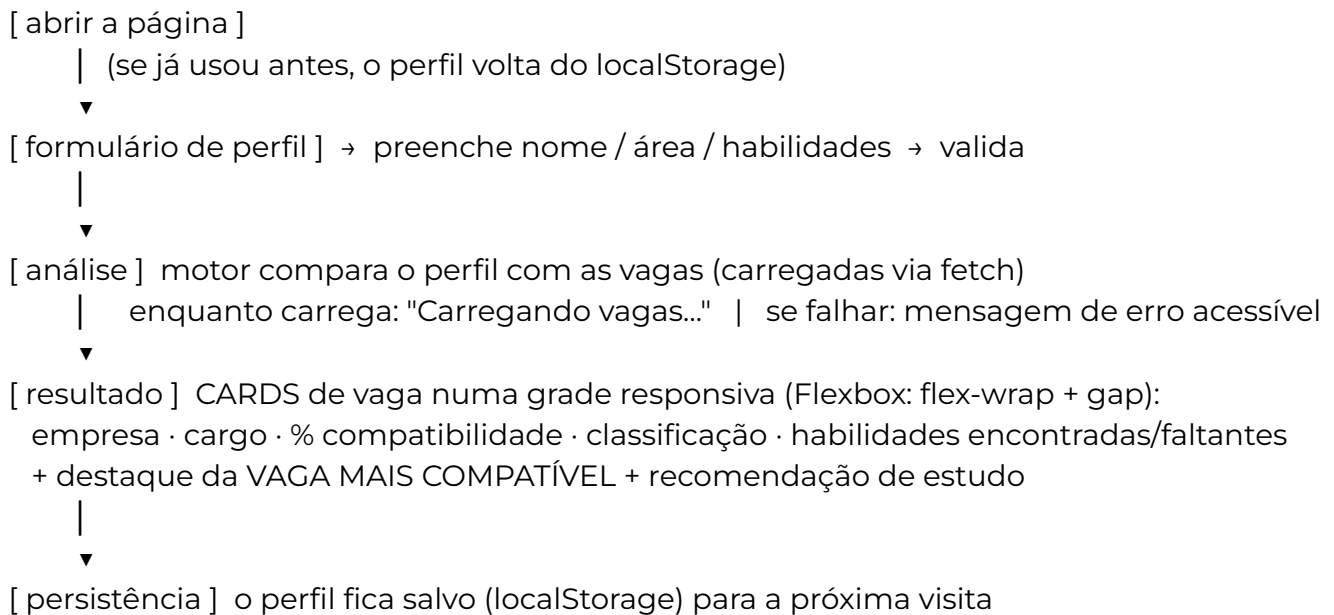
No vídeo, seja honesto sobre onde usou IA e o que você mudou/validou. Isso conta a favor, mostra maturidade profissional, que é exatamente o que o mercado paga.

5. ROTEIRO DA APLICAÇÃO

Abaixo segue um detalhamento de como a aplicação deve se comportar (exemplos de telas, modelagem de sistemas, banco de dados, etc.)

5.1. FORMATO DO SISTEMA

A aplicação é uma página web única (Single Page) em HTML, CSS e JavaScript puro, rodando no navegador (via Live Server). O fluxo do usuário:



Telas/áreas mínimas: (1) cabeçalho/identidade; (2) formulário de perfil; (3) área de resultados (cards + destaque da melhor vaga + recomendação); (4) rodapé. Tudo semântico, responsivo e acessível. Você pode esboçar isso no Excalidraw antes de codar (como fizemos em aula).

5.2. DOCUMENTAÇÃO NO README.MD

Crie um arquivo `readme.md` no repositório do seu projeto no GitHub, para documentar a sua solução, bem como demonstrar as técnicas e linguagens utilizadas, além do escopo do projeto e como o usuário pode executar o seu sistema.

Algumas dicas interessantes para utilizar na criação do seu portfólio são:

- Criar um nome para o seu software;
- Descrever qual o problema ele resolve;
- Descrever quais técnicas e tecnologias utilizadas. Aqui você também pode inserir alguma imagem ou diagrama para melhor entendimento;
- Descrever como executar;
- Descrever quais melhorias podem ser aplicadas;
- Entre outras coisas.

5.3. USO DO GITHUB

Utilizar um repositório GitHub para controle de versionamento do projeto. Siga as diretrizes de boas práticas de uso do Git abaixo:

- Seguir *commits* com descrição sucinta e direta e, de preferência, com poucos arquivos alterados por *commit*.
 - Prefira mensagens como “implementa X” ou “corrige Y”, em vez de “X foi implementado” ou “Y foi corrigido”.

- Crie uma *branch* “develop” para concentrar os merges enquanto estiver desenvolvendo as funcionalidades.
- Utilizar *feature branches* a partir da *develop* para cada tarefa.
 - Não excluir a *branch* após o pull request concluído.
- O código final do projeto deve estar todo “mergeado” no *branch* “main”.

5.4. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento deste sistema você deverá gravar um vídeo, com tempo **máximo de 7 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?
- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link, e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no *readme.md* do seu projeto no repositório do GitHub.

6. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 60% sobre a avaliação do módulo.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Apresentação do Projeto				
Nº	Critério de Avaliação	0	1,00	2,00
1	Realizou a gravação de um vídeo?	Não foi realizada a gravação do vídeo.	Gravou o vídeo e abordou parte dos tópicos listados no item 5.4.	Gravou o vídeo e abordou todos os tópicos listados no item 5.4.

Uso do GitHub e Readme.md				
Nº	Critério de Avaliação	0	0,25	1,00
2	Versionamento com branches e commits	O repositório do projeto não apresenta branches e commits.	O repositório do projeto apresenta parte das branches e commits distintos e nomeadas padronizadamente para cada	O repositório do projeto apresenta branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.

			funcionalidade desenvolvida.	
3	Organização dos arquivos no repositório	O repositório do projeto não apresenta os arquivos (estrutura de pastas e README.md) estruturados conforme as instruções.	O repositório do projeto apresenta parte dos arquivos (estrutura de pastas e README.md) estruturados conforme as instruções.	O repositório do projeto apresenta os arquivos (estrutura de pastas e README.md) estruturados conforme as instruções.

Desenvolvimento da Aplicação

Nº	Critério de Avaliação	0	0,25	0,50
4	Modelagem do perfil do candidato e catálogo de vagas	Não modelou o perfil nem o catálogo de vagas.	Modelou apenas um dos dois, ou em formato incompleto.	Modelou o perfil do candidato em objeto e o catálogo de vagas estruturado como uma coleção (array) de objetos.
5	Compatibilidade entre candidato e vagas	Não calculou a compatibilidade entre o candidato e as vagas.	Calculou a compatibilidade com erros, ou deixou de classificar as vagas ou de apontar a melhor vaga com a recomendação.	Implementou o cálculo da compatibilidade (habilidades encontradas e faltantes), de classificação das vagas em Alta/Média/Baixa e apontou a melhor vaga com a recomendação de estudo.
6	Utilização de métodos	Não utilizou métodos de array, ou utilizou apenas um.	Utilizou dois métodos de array.	Utilizou pelo menos três métodos de arrays (map, filter, find, every, reduce) para realizar a filtragem e a transformação de listas.
7	Utilização de métodos de classe	Não criou nenhuma classe.	Criou a classe, mas não incluiu a herança, ou criou a herança sem novas funcionalidades.	Criou a classe com this e herança justificada, em que a subclasse acrescenta ou sobrescreve comportamento.
8	Emprego de callback e closure	Não empregou callback nem closure.	Empregou apenas um dos dois.	Empregou closures para a preservação do estado de variáveis locais e funções de callback para o controle do fluxo de execução.
9	Estrutura da página HTML	Estruturou a página sem semântica ("div-soup") e sem acessibilidade.	Utilizou tags semânticas, mas falhou em itens de acessibilidade ou de SEO.	Utilizou landmarks e um único h1, associou rótulos aos campos, usou alt e foco visível, e cuidou do SEO on-page na estrutura da página HTML
10	Responsividade	A aplicação quebrou no celular.	Deixou a aplicação parcialmente responsiva, com poucos breakpoints.	Aplicou responsividade com Flexbox, unidades fluidas e <i>media queries</i> , funcionando em

				dispositivos móveis e desktop.
11	Formulário e validação	Não construiu o formulário, ou não tratou os eventos.	Construiu o formulário, mas sem validação ou sem preventDefault.	Construiu o formulário com addEventListener, preventDefault e validação com mensagem de erro acessível.
12	Conteúdo dinâmico	Deixou os cards fixos no HTML.	Gerou os cards parcialmente, misturando HTML fixo e JavaScript.	Gerou os cards inteiramente por JavaScript, a partir dos dados.
13	Consumo de dados	Não utilizou fetch.	Utilizou fetch, mas não tratou os três estados.	Utilizou fetch com try/catch e response.ok, tratando os estados de carregando, vazio e erro.
14	LocalStorage	Não persistiu nenhum dado.	Persistiu os dados, mas quebrou no null da primeira visita ou não usou JSON.	Persistiu os dados com JSON.stringify/parse e tratou o null da primeira visita.
15	Módulos ES	Manteve todo o código em um único arquivo, sem evidência de depuração.	Organizou em módulos ES, mas utilizou uma versão menos atual.	Separou o código em módulos ES.
16	Código limpo	Não comentou e não usou nomenclatura condizente ao contexto	Pouco ou excesso de comentários e nomenclaturas parcialmente relacionada	Criou variáveis, funções, classes e ids com nomes contextuais e comentou trechos relevantes com explicações condizentes ao código

7. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

- ☐ Repositório privado no GitHub, com mentor/operação adicionados
- ☐ A aplicação roda no Live Server (HTML + CSS + JS)
- ☐ Motor do SkillMatch reaproveitado: compatibilidade, faltantes, classificação, melhor vaga, recomendação
- ☐ ≥3 métodos de array; POO com classe + herança + this; callback e closure
- ☐ HTML semântico + acessível (landmarks, um h1, label/for, alt, foco, lang) + SEO (title, meta description)
- ☐ Formulário com validação e eventos; cards gerados por JavaScript (DOM)
- ☐ Responsivo mobile-first (testado no modo responsivo do DevTools)
- ☐ fetch das vagas com os 3 estados (carregando/vazio/erro)
- ☐ localStorage lembrando do perfil (com JSON e tratamento de null)
- ☐ JavaScript em módulos ES (motor/ui/dados)
- ☐ Nada fora do escopo (sem React/TS/build/back-end/Grid de layout), conferi o que veio da IA
- ☐ Branches + commits descritivos (≥8) mergeados na main

- ☐ Trello público com os cartões + link no README
- ☐ README completo (5.2)
- ☐ Vídeo (≤ 7 min) no Google Drive com permissão por link
- ☐ Links enviados no AVA antes do prazo